



LibraryOS

Setup & Configuration Guide

Windows • macOS • Linux

Covers: prerequisites, installation, environment configuration, database setup, running the app, testing, scheduled reminders, and production deployment — with Windows-specific commands throughout.

1. What this guide covers

LibraryOS is a Flask + PostgreSQL library management app. This guide walks through setting it up from a clean machine to a running app, with special attention to **Windows**, where a few commands (shell activation, `cp`, `cron`, `Gunicorn`) differ from the Linux/macOS instructions in the project's `README.md`. Linux/macOS users can follow this guide too — Windows-only steps are marked.

2. Prerequisites

Requirement	Windows	macOS / Linux
Python 3.10+	Install from <code>python.org</code> or <code>winget install Python.Python.3.12</code> . Check "Add python.exe to PATH" during setup.	Usually preinstalled; otherwise <code>brew install python3</code> / your distro's package manager.
Git	<code>winget install Git.Git</code> , or <code>git-scm.com</code>	<code>brew install git</code> / <code>apt install git</code>
PostgreSQL 14+	Either the native installer from <code>postgresql.org/download/windows</code> (includes <code>psql</code> and <code>pgAdmin</code>), or Docker Desktop running a <code>postgres</code> container (recommended — matches this project's own dev setup).	Native package, or Docker.
Docker Desktop (optional but recommended)	<code>docker.com/products/docker-desktop</code> — requires WSL2, which the installer sets up for you if missing.	Docker Desktop, or your distro's <code>docker</code> package.

Recommended path: use Docker Desktop for Postgres even on Windows. It avoids the native Windows Postgres installer's service-account quirks entirely, and the exact same `docker run` command works identically on Windows, macOS, and Linux.

3. Get the code

```
git clone https://github.com/iHiteshShibag/library-management-flask.git
cd library-management-flask
```

4. Create and activate a virtual environment

Windows — Command Prompt

```
python -m venv venv
venv\Scripts\activate.bat
```

Windows — PowerShell

```
python -m venv venv
venv\Scripts\Activate.ps1
```

PowerShell blocking the script? PowerShell's default execution policy blocks unsigned scripts, including the venv activation script. Fix it for the current session only (safe, doesn't change system-wide settings):

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Then re-run `venv\Scripts\Activate.ps1` .

macOS / Linux

```
python3 -m venv venv
source venv/bin/activate
```

Your prompt should now be prefixed with `(venv)` on any platform — that confirms the virtual environment is active before you install anything.

5. Install dependencies

```
pip install -r requirements.txt
```

For running the test suite too, install the dev extras instead (it includes everything above, plus `pytest`):

```
pip install -r requirements-dev.txt
```

`psycopg2-binary` (the Postgres driver) ships precompiled wheels for Windows, so this step needs no separate C compiler or build tools on any platform.

6. Start PostgreSQL

Option A — Docker (recommended, identical on every OS)

```
docker run --name library-pg -e POSTGRES_USER=library ^  
  -e POSTGRES_PASSWORD=library_dev_pw -e POSTGRES_DB=library ^  
  -p 5432:5432 -d postgres:16
```

(The `^` line-continuation above is Command Prompt syntax. In PowerShell use a backtick ``` instead; in a single line on macOS/Linux, just remove the continuations and use `\` or write it as one line.)

Option B — Native Windows PostgreSQL install

1. Run the installer from [postgresql.org](https://www.postgresql.org); note the password you set for the `postgres` superuser.
2. Open **SQL Shell (psql)** from the Start menu, connect as `postgres`, then create the app's role and database:

```
CREATE USER library WITH PASSWORD 'library_dev_pw';  
CREATE DATABASE library OWNER library;
```

7. Configure environment variables

Windows — Command Prompt

```
copy .env.example .env
```

Windows — PowerShell

```
Copy-Item .env.example .env
```

macOS / Linux

```
cp .env.example .env
```

Open `.env` in any text editor (Notepad is fine) and set:

Variable	Required?	What it's for
<code>SECRET_KEY</code>	Yes	Flask session signing. Generate with: <pre>python -c "import secrets; print(secrets.token_hex(32))"</pre>
<code>DATABASE_URL</code>	Yes	Postgres connection string, e.g. <code>postgresql://library:library_dev_pw@localhost:5432/library</code>
<code>FLASK_DEBUG</code>	No (default <code>false</code>)	Set <code>true</code> only for local debugging — never in production.
<code>EMAIL_USER</code> / <code>EMAIL_PASSWORD</code>	No	SMTP credentials for real password-reset/reminder emails. Omit to have them logged/flashed instead (fine for local testing).
<code>EMAIL_SMTP_HOST</code> / <code>EMAIL_SMTP_PORT</code>	No (default Gmail)	SMTP server details, if not using Gmail.
<code>SESSION_COOKIE_SECURE</code>	No (default <code>false</code>)	Set <code>true</code> only once the app is served over HTTPS — otherwise login breaks over plain <code>http://</code> .
<code>RATELIMIT_STORAGE_URI</code>	No (default in-memory)	Point at Redis (e.g. <code>redis://localhost:6379</code>) if running more than one worker process.
<code>SENTRY_DSN</code>	No	Optional error tracking. Leave unset to skip Sentry entirely.

8. Run database migrations

```
flask db upgrade
```

This creates all tables (`organizations` , `users` , `books` , `members` , `transactions` , `reservations` , `audit_logs`) in the database from step 6.

9. Run the app

```
python app.py
```

Open **`http://127.0.0.1:5000`** in your browser. Sign up once to found the organization and become its admin — every signup after the first joins that same organization as staff instead (single-org mode). Alternatively, create the first admin from the command line:

```
flask create-admin
```

Windows Firewall prompt: the first time you run the app, Windows may ask whether to allow Python through the firewall. Since the app only binds to `127.0.0.1` (your own machine) by default, this is safe to allow or deny — either way, nothing outside your machine can reach it.

10. Schedule due-date reminders

The app has no built-in job scheduler — `flask send-due-reminders` emails members whose loans are approaching or past the grace period, and needs to be triggered externally, once a day.

Windows — Task Scheduler

1. Open **Task Scheduler** → *Create Basic Task*.
2. Trigger: *Daily*, pick a time (e.g. 7:00 AM).
3. Action: *Start a program*. Program/script: the full path to `venv\Scripts\python.exe`. Arguments: `-m flask --app app send-due-reminders`. Start in: the project folder path.

macOS / Linux — cron

```
0 7 * * * cd /path/to/library-management-flask && venv/bin/flask send-due-reminders
```

11. Running the test suite

Tests run against a **real Postgres database**, not SQLite — the dashboard's analytics queries use Postgres-only SQL (`date_trunc`), which SQLite doesn't support, so a SQLite-backed test run would silently hide real bugs instead of catching them.

Create a dedicated test database once (same `psql` / Docker approach as step 6):

```
CREATE DATABASE library_test OWNER library;
```

Then, with the dev extras installed (step 5):

```
pytest
```

It defaults to `postgresql://library:library_dev_pw@localhost:5432/library_test` — override with the `TEST_DATABASE_URL` environment variable if your setup differs.

12. Production deployment

Windows cannot run Gunicorn. Gunicorn depends on `os.fork`, which does not exist on Windows — it will fail to start there under any circumstances, not just "not recommended." Use one of these instead:

Option A — Waitress (pure-Python, works natively on Windows)

```
pip install waitress
waitress-serve --host=0.0.0.0 --port=8000 app:app
```

Option B — WSL2 or Docker (run the real Linux stack)

Inside WSL2 or a Linux container, follow the Linux instructions and use Gunicorn normally:

```
gunicorn app:app --workers 4
```

Either way, put a reverse proxy in front to terminate HTTPS — IIS with URL Rewrite/ARR, Nginx (available on Windows too), or Caddy all work. A minimal Nginx example:

```
server {
    listen 443 ssl;
    server_name your-domain.example;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Production checklist (any OS)

- Serve behind HTTPS via a reverse proxy (never expose the dev server or a bare WSGI server directly).
- Set `SESSION_COOKIE_SECURE=true` only once HTTPS is actually in front of the app.
- Generate a fresh `SECRET_KEY` and a strong, unique Postgres password for this environment — don't reuse local dev values.
- Set real `EMAIL_USER` / `EMAIL_PASSWORD` (or another SMTP-speaking provider) so password resets and reminders actually get delivered.
- Point `RATELIMIT_STORAGE_URI` at Redis if running more than one worker/process.
- Optionally set `SENTRY_DSN` for error tracking.

13. Troubleshooting (Windows-specific)

Symptom	Fix
<code>'python' is not recognized...</code>	Python wasn't added to PATH during install. Re-run the installer and check "Add python.exe to PATH", or use the full path to <code>python.exe</code> .
<code>venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled</code>	Run <code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass</code> first (see step 4).
<code>psql: command not found</code>	Either use <code>docker exec -it library-pg psql -U library -d library</code> instead (if using Docker), or add PostgreSQL's <code>bin</code> folder to PATH (native install).
App won't start, port 5000 already in use	Another process (sometimes Windows' own service) holds port 5000. Run on a different port: <code>flask --app app run --port 5050</code> .
<code>gunicorn: command not found</code> / immediate crash	Expected — Gunicorn doesn't support Windows at all. Use Waitress or WSL2/Docker (step 12).